```
(
    [0] => ASCII
    [1] => UTF-8
)
```

There is one setting that has no `.ini` file equivalent nor a corresponding function: `illegal_chars`. There is no official documentation for this, and I had to delve into the C source code to figure out what it's used for. `illegal_chars` is initialized on program startup to an unsigned integer with a value of *0*. It is used (very inconsistently) to keep a count of invalid byte sequences by just a handful of functions. The following code demonstrates how it can be used:

```
$invalid2Octet = "\xC3\x28";
$invalidSequenceId = "\xA0\xA1";

print mb_get_info('illegal_chars') . PHP_EOL;
$converted = mb_convert_encoding(
    "I am {$invalid2Octet}} broken {$invalidSequenceId}",
    'UTF-16BE',
    'UTF-8'
);
print $converted . PHP_EOL;
print bin2hex($converted) . PHP_EOL;
print mb_get_info('illegal_chars') . PHP_EOL;

Output:
0
I am ?( broken ??
004900200061...
3
```

The three invalid byte sequences (*0xC3*, *0xA0*, and *0xA1–0x28*, which is valid on its own as an opening-parenthesis (character) are substituted for the ASCII (decimal) ordinal value *63* (the sixty-third character: a question mark). The `illegal_chars` counter is incremented accordingly. The value will persist between calls; this way it is possible to keep track of how many illegal bytes are found as the program does its work. I can say with some certainty that `mb_convert_encoding()` and `mb_output_handler()` are the